

VIT-AP
UNIVERSITY

DIGITAL LOGIC DESIGN
(Course Code: ECE 1003)

Module-1:Lecture-13

Boolean Algebra

CONTENTS

Module-1 (Part-2)

- ❖ Logic gates and universal gates
- ❖ Tri-state logic
- ❖ Boolean algebra
- ❖ SOP and POS minimization
- ❖ **K-maps upto 4-variables**
- ❖ Don't care logic

CONTENTS

Lecture-13

- ❖ Gate Level Minimization
 - POS representation
- ❖ NAND implementation
- ❖ NOR implementation

BOOKS

Textbooks

1. M.Morris Mano, Michael D Ciletti, **Digital Design**, 5th edition, Pearson Publishers, 2013.
2. R.P. Jain, “**Modern Digital Electronics**”, 4th edition, TMH.

References

1. M.Morris Mano, Charles R. Kime, Tom Martin, **Logic and Computer Design Fundamentals**, 4th edition, Pearson Publishers.
2. C. H. Roth and L. L. Kinney, **Fundamentals of Logic Design**, 5th edition, Cengage Publishers.

Gate Level Minimization

PRODUCT OF SUMS (POS) REPRESENTATION USING MAXTERMS

3-Variable K-Map:

		yz			
		$y + z$	$y + z'$	$y' + z'$	$y' + z$
		00	01	11	10
x	0	M_0 $(x + y + z)$	M_1 $(x + y + z')$	M_3 $(x + y' + z')$	M_2 $(x + y' + z)$
	1	M_4 $(x' + y + z)$	M_5 $(x' + y + z')$	M_7 $(x' + y' + z')$	M_6 $(x' + y' + z)$

PRODUCT OF SUMS (POS) REPRESENTATION USING MAXTERMS

4-Variable K-Map:

		yz			
		$y + z$	$y + z'$	$y' + z'$	$y' + z$
		00	01	11	10
WX	$w + x$ 00	M_0 $(w + x + y + z)$	M_1 $(w + x + y + z')$	M_3 $(w + x + y' + z')$	M_2 $(w + x + y' + z)$
	$w + x'$ 01	M_4 $(w + x' + y + z)$	M_5 $(w + x' + y + z')$	M_7 $(w + x' + y' + z')$	M_6 $(w + x' + y' + z)$
	$w' + x'$ 11	M_{12} $(w' + x' + y + z)$	M_{13} $(w' + x' + y + z')$	M_{15} $(w' + x' + y' + z')$	M_{14} $(w' + x' + y' + z)$
	$w' + x$ 10	M_8 $(w' + x + y + z)$	M_9 $(w' + x + y + z')$	M_{11} $(w' + x + y' + z')$	M_{10} $(w' + x + y' + z)$

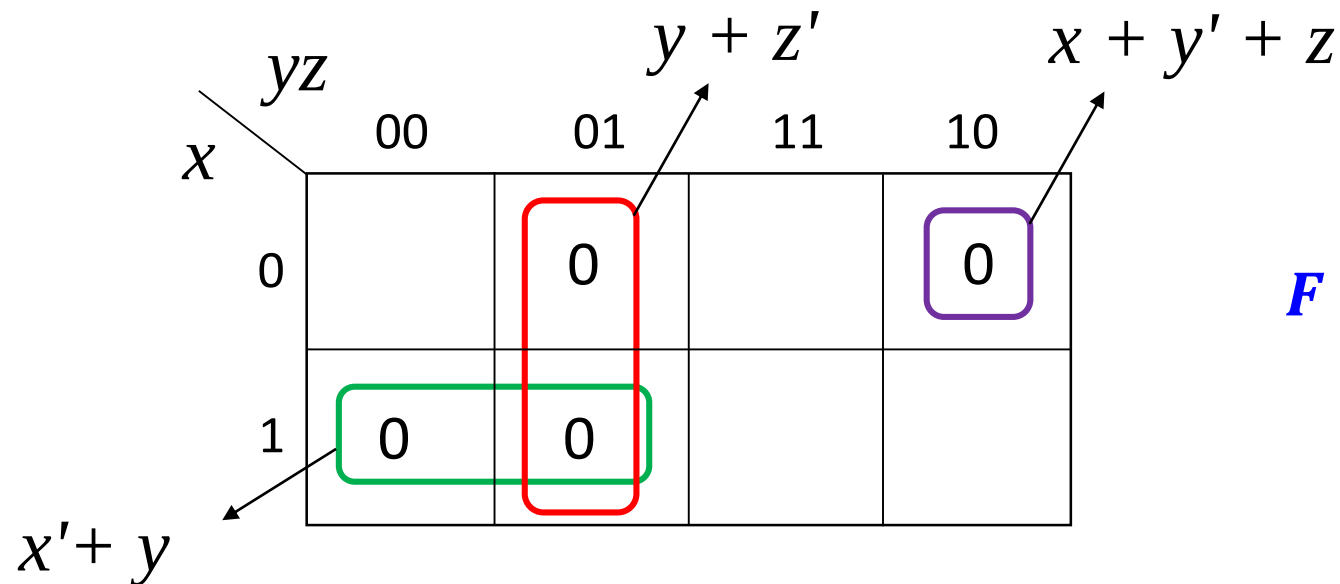
PRODUCT OF SUMS (POS) REPRESENTATION USING MAXTERMS

Example: Simplify the following Boolean function into Product of sums (POSs).

i. $F = \Pi(1, 2, 4, 5)$

ii. $F = \Pi(1, 2, 3, 8, 9, 10, 11, 14)$

Solution: i) The **POS form** can be obtained as

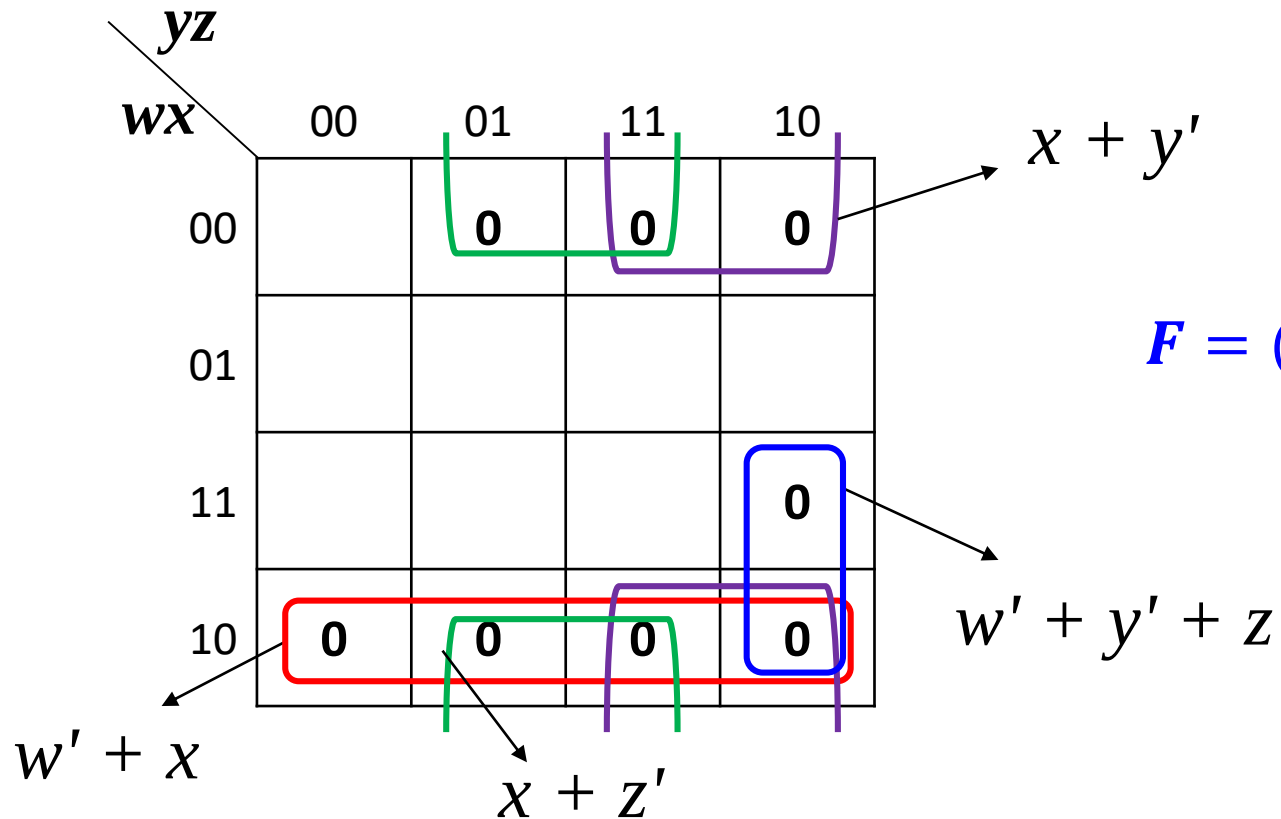


$$F = (x' + y)(y + z')(x + y' + z)$$

PRODUCT OF SUMS (POS) REPRESENTATION USING MAXTERMS

Solution: ii) The **SOP form** can be obtained as

$$F = \Pi(1, 2, 3, 8, 9, 10, 11, 14)$$



$$F = (x + y')(w' + x)(x + z')(w' + y' + z)$$

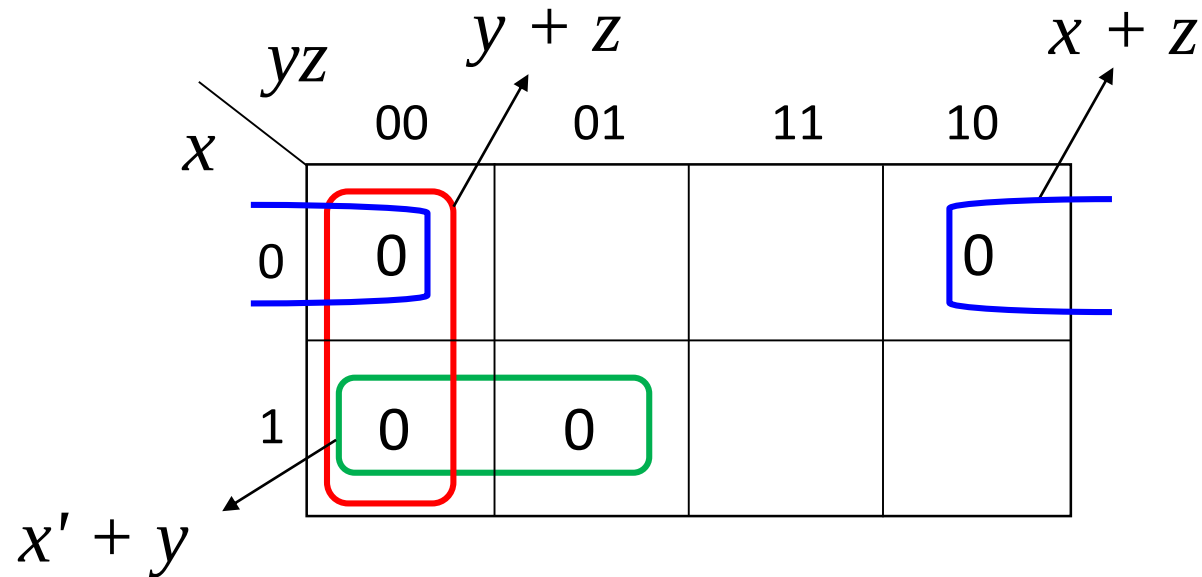
PRODUCT OF SUMS (POS) REPRESENTATION USING MAXTERMS

Example: Simplify the following Boolean function into both POS and SOP.

i. $F = \Pi(0, 2, 4, 5)$

ii. $F = \Pi(0, 1, 2, 4, 5, 9, 10, 12, 14)$

Solution: i) The **POS form** can be obtained as



$$F = (x' + y)(y + z)(x + z)$$

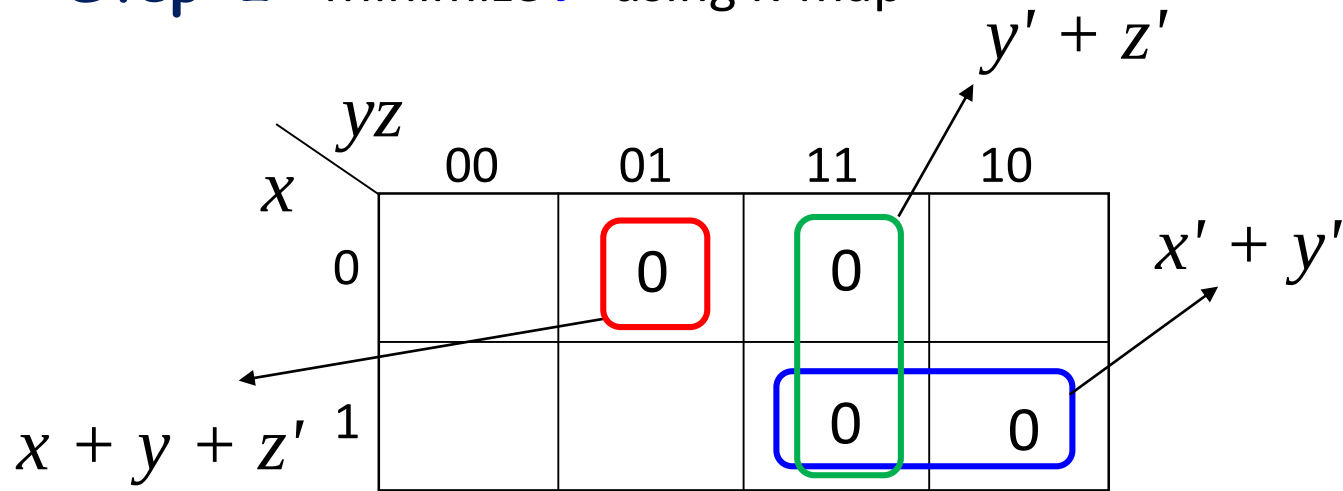
PRODUCT OF SUMS (POS) REPRESENTATION USING MAXTERMS

Solution: i) The **SOP form** can be obtained as

Step-1: Obtain the complement of F

$$F' = \Pi(1, 3, 6, 7)$$

Step-2: Minimize F' using K-map



$$F' = (x' + y')(y' + z')(x + y + z')$$

Step-3: Obtain F using De 'Morgan's Law

$$F = (F')' = ((x' + y')(y' + z')(x + y + z'))'$$

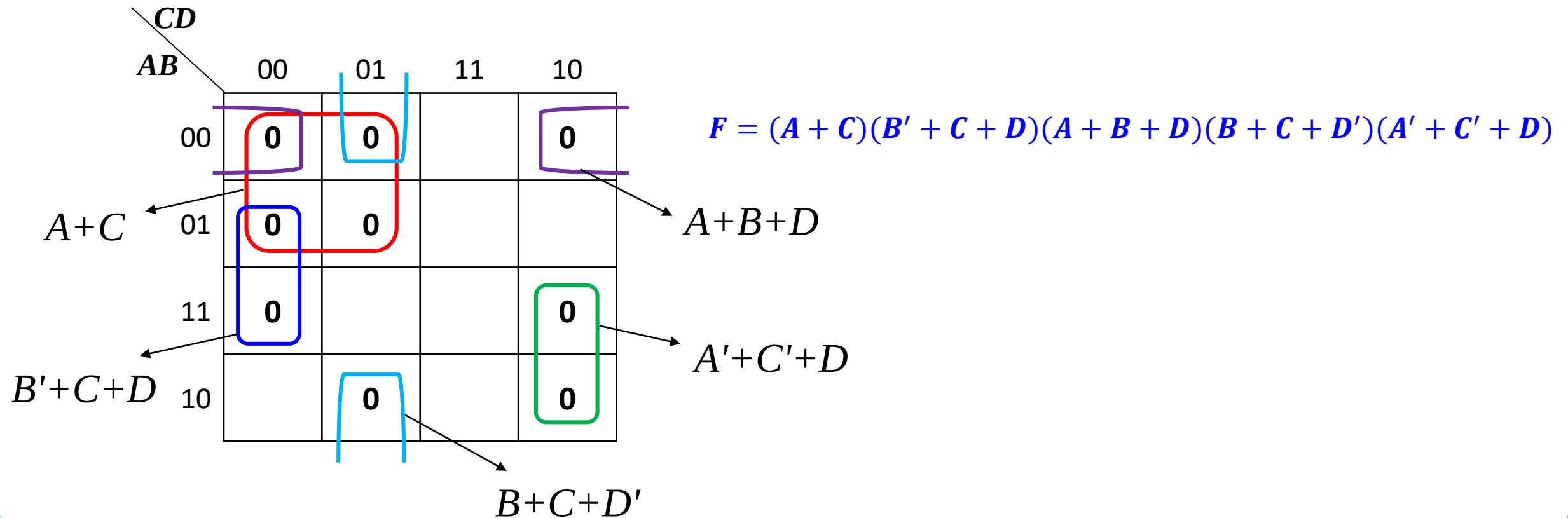
$$F = (x' + y')' + (y' + z')' + (x + y + z')'$$

$$F = xy + yz + x'y'z$$

PRODUCT OF SUMS (POS) REPRESENTATION USING MAXTERMS

Solution: ii) The **POS form** can be obtained as

$$F = \Pi(0, 1, 2, 4, 5, 9, 10, 12, 14)$$

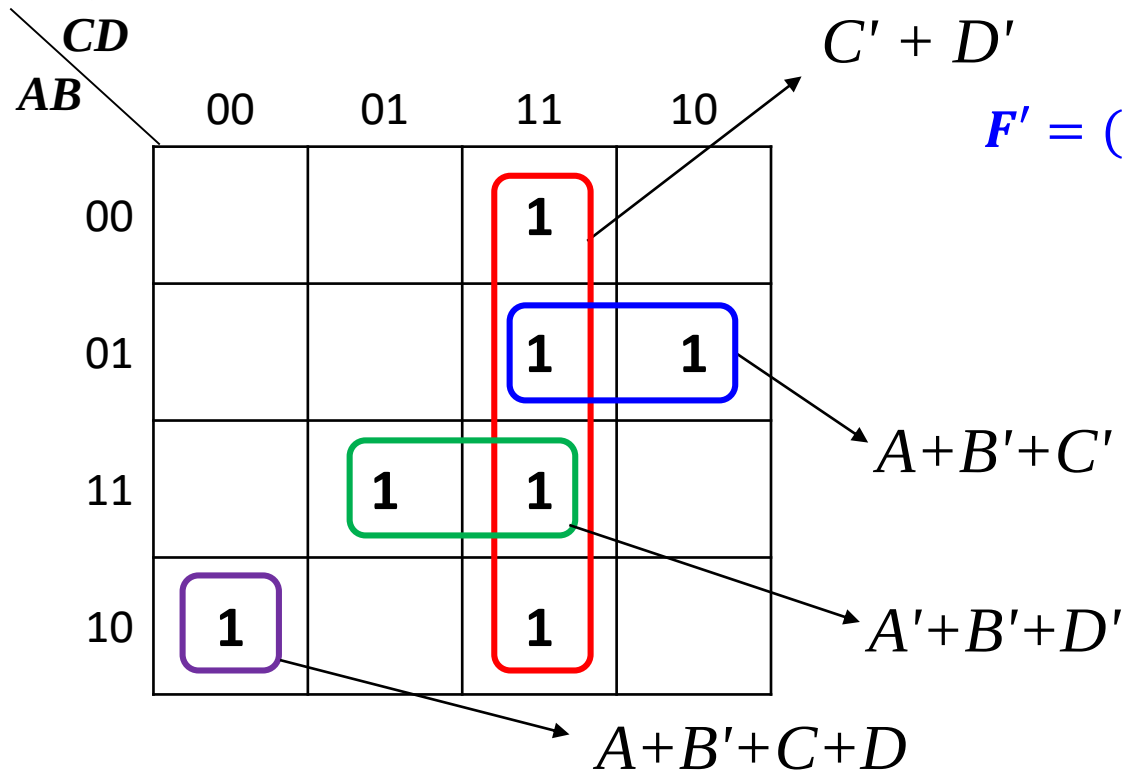


PRODUCT OF SUMS (POS) REPRESENTATION USING MAXTERMS

Solution: ii) The **SOP form** can be obtained as

Step-1: Obtain the complement of F : $F' = \Pi(3, 6, 7, 8, 11, 13, 15)$

Step-2: Minimize F' using K-map



$$F' = (C' + D')(A + B' + C')(A' + B' + D')(A + B' + C + D)$$

Step-3: Obtain F using De 'Morgan's Law

$$F = (F')' = [(C' + D')(A + B' + C')(A' + B' + D')(A + B' + C + D)]'$$

$$F = (C' + D')' + (A + B' + C')' + (A' + B' + D')' + (A + B' + C + D)'$$

$$F = CD + A'BC + ABD + A'BC'D'$$

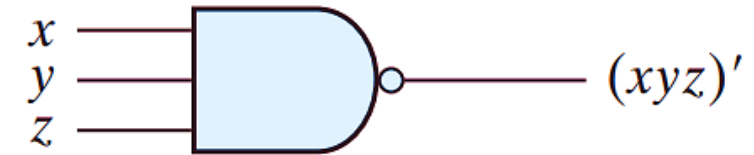
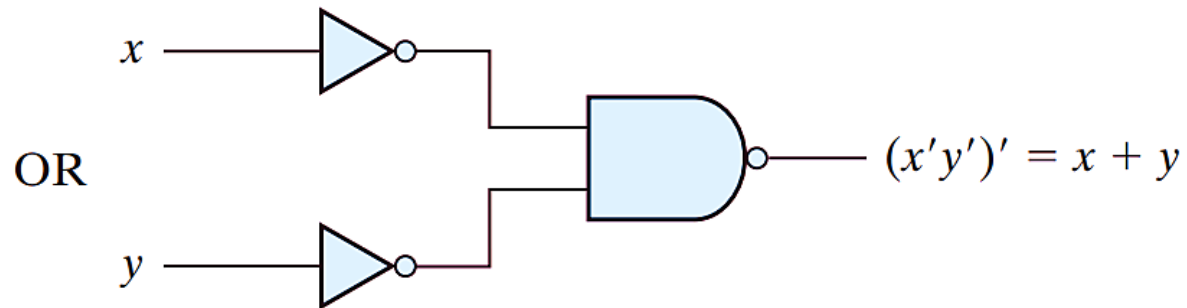
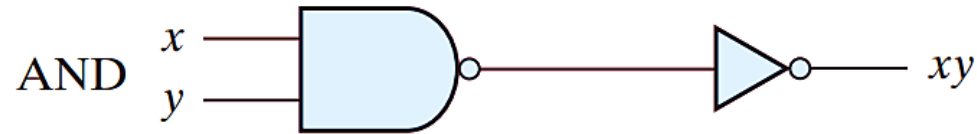
UNIVERSAL GATES

Universal Gates

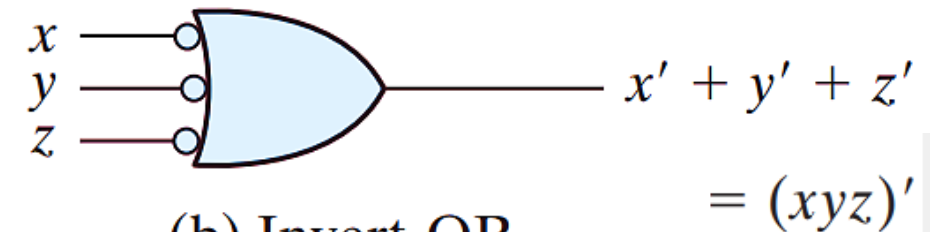
- ✓ The **NAND and NOR gates** are said to be **universal gates** because any logic circuit can be implemented with it.
- ✓ To show that any Boolean function can be implemented with universal gates, we need only show that the basic logical operations of AND, OR, and complement can be obtained with universal gates alone.

NAND IMPLEMENTATION

- ✓ Any Boolean function, we need to show the basic logical operations, i.e., AND, OR and NOT operations in terms NAND alone.



(a) AND-invert



(b) Invert-OR

**Both (a) and (b) will perform
NAND operation**

NAND IMPLEMENTATION

Method-1:

- ✓ Simplify the Boolean function into sum of products (**SOP**) form.
- ✓ Represent F in $(F')'$ form. Now each product term can be represented as NAND operation.

Example-1: Implement the following Boolean function using NAND gates

$$F = AB + CD$$

Solution:

Step-1: The function already in **SOP form**

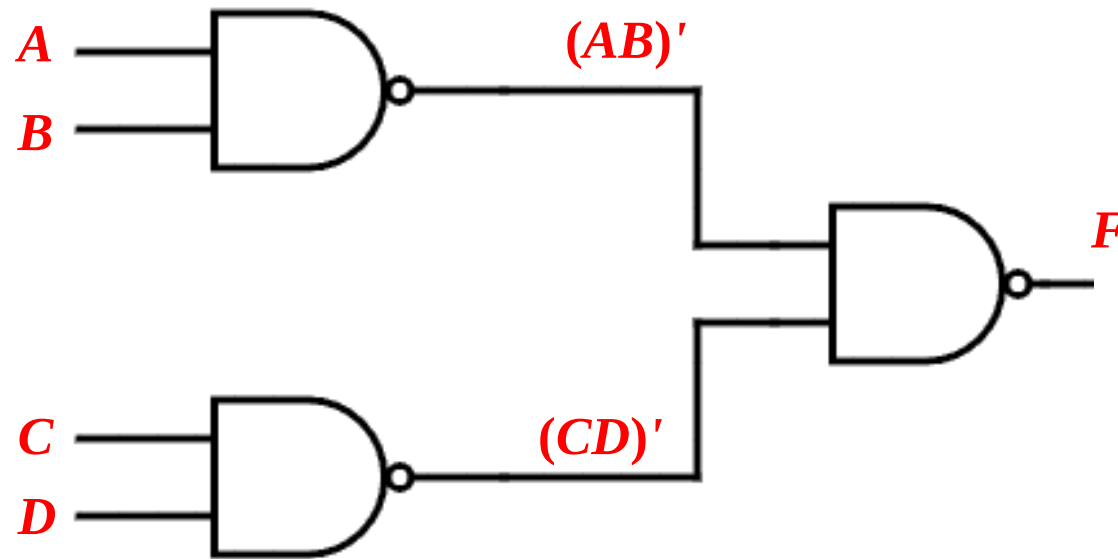
Step-2:
$$F = (F')' = [(AB + CD)']' = [(AB)'(CD)']' = [xy]'$$

where, $x = (AB)'$ & $y = (CD)'$

NAND IMPLEMENTATION

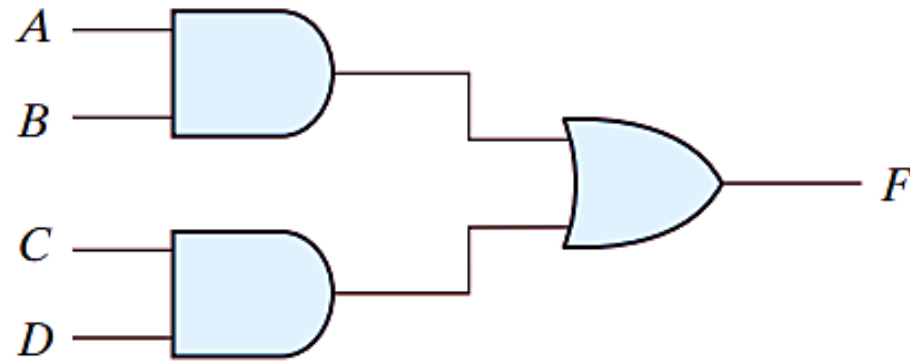
Example-1:

Step-3: Here, $(AB)'$ represents a NAND gate
 $(CD)'$ represents a NAND gate
 $[xy]'$ represents a NAND gate

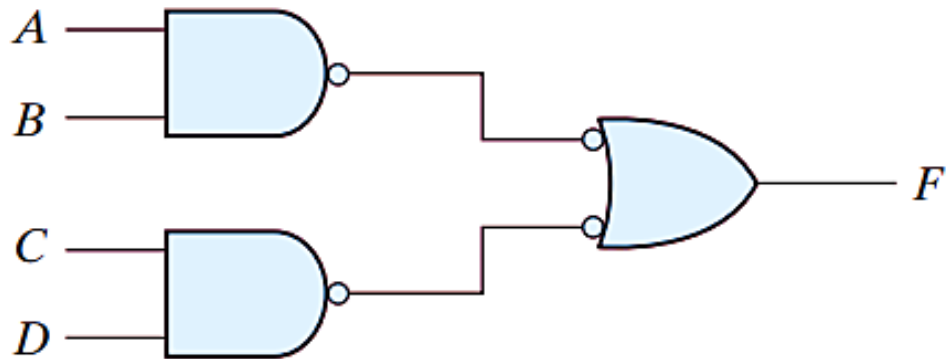


NAND IMPLEMENTATION

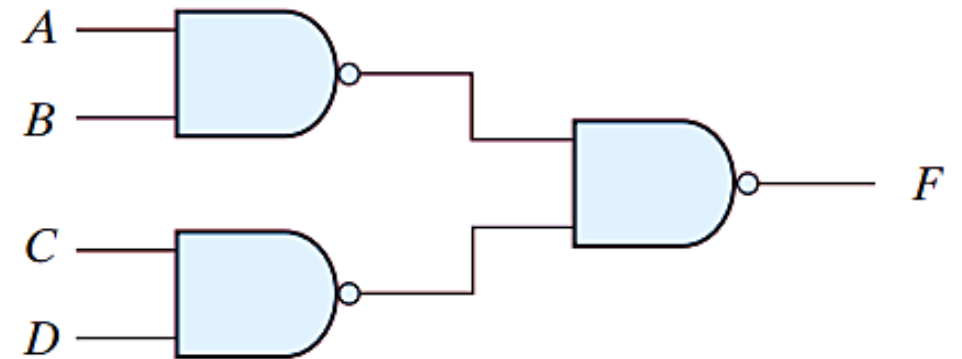
Method-2:



(a)



(b)



(c)

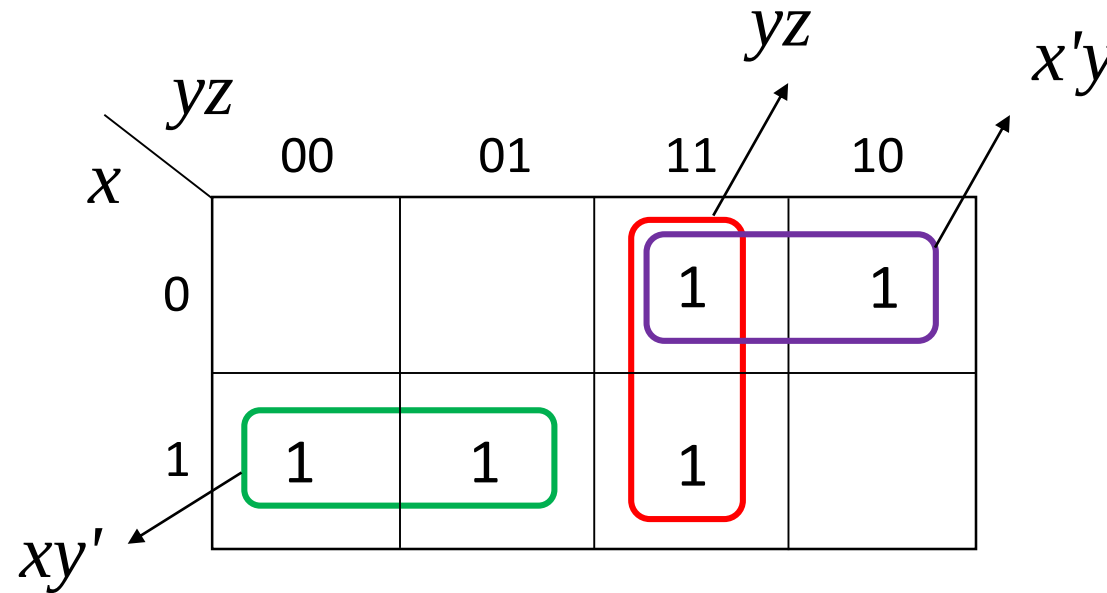
NAND IMPLEMENTATION

Example-2: Implement the following Boolean function using NAND gates

$$F = \Sigma(2, 3, 4, 5, 7)$$

Solution:

Step-1: Obtain the SOP form



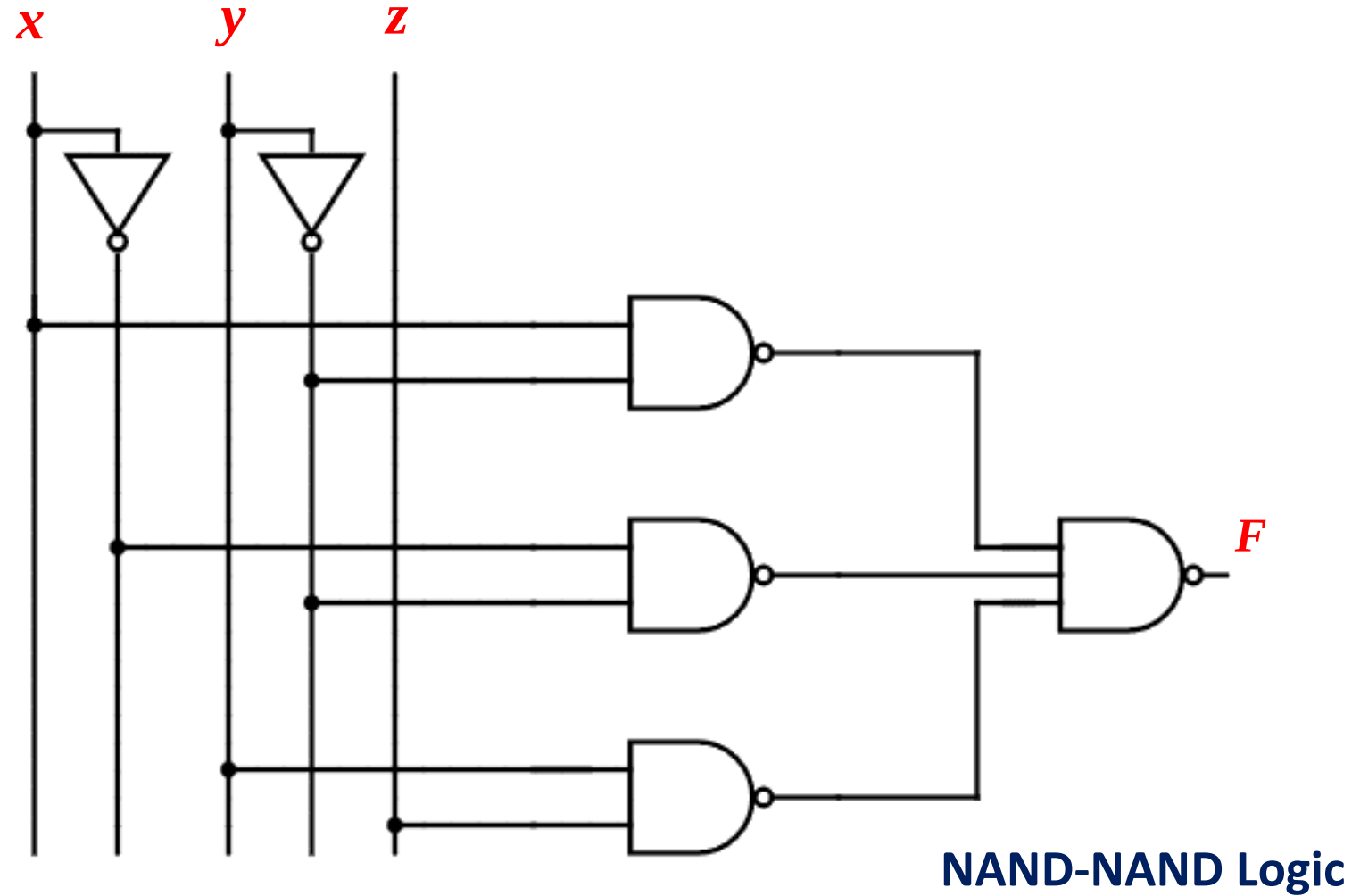
$$F = xy' + x'y + yz$$

Step-2:

$$F = (F')' = [(xy' + x'y + yz)']' = [(xy')'(x'y)'(yz)']'$$

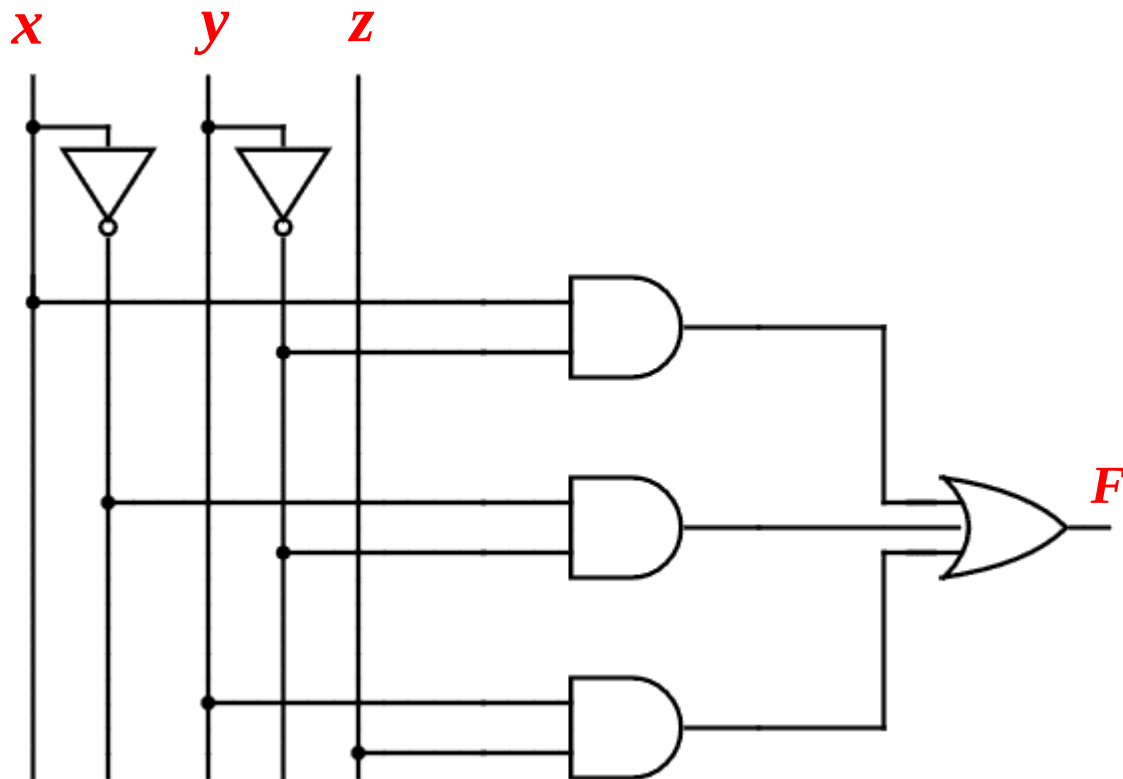
NAND IMPLEMENTATION

Step-3:

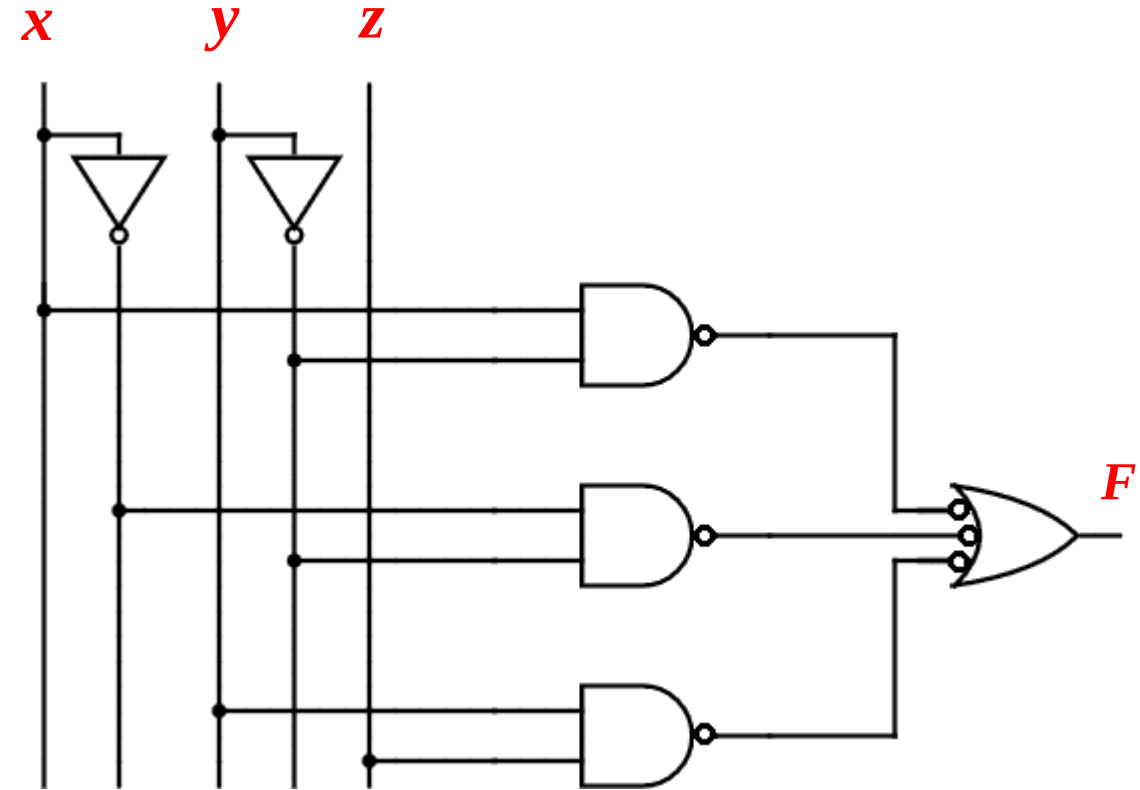


NAND IMPLEMENTATION

Method-2:



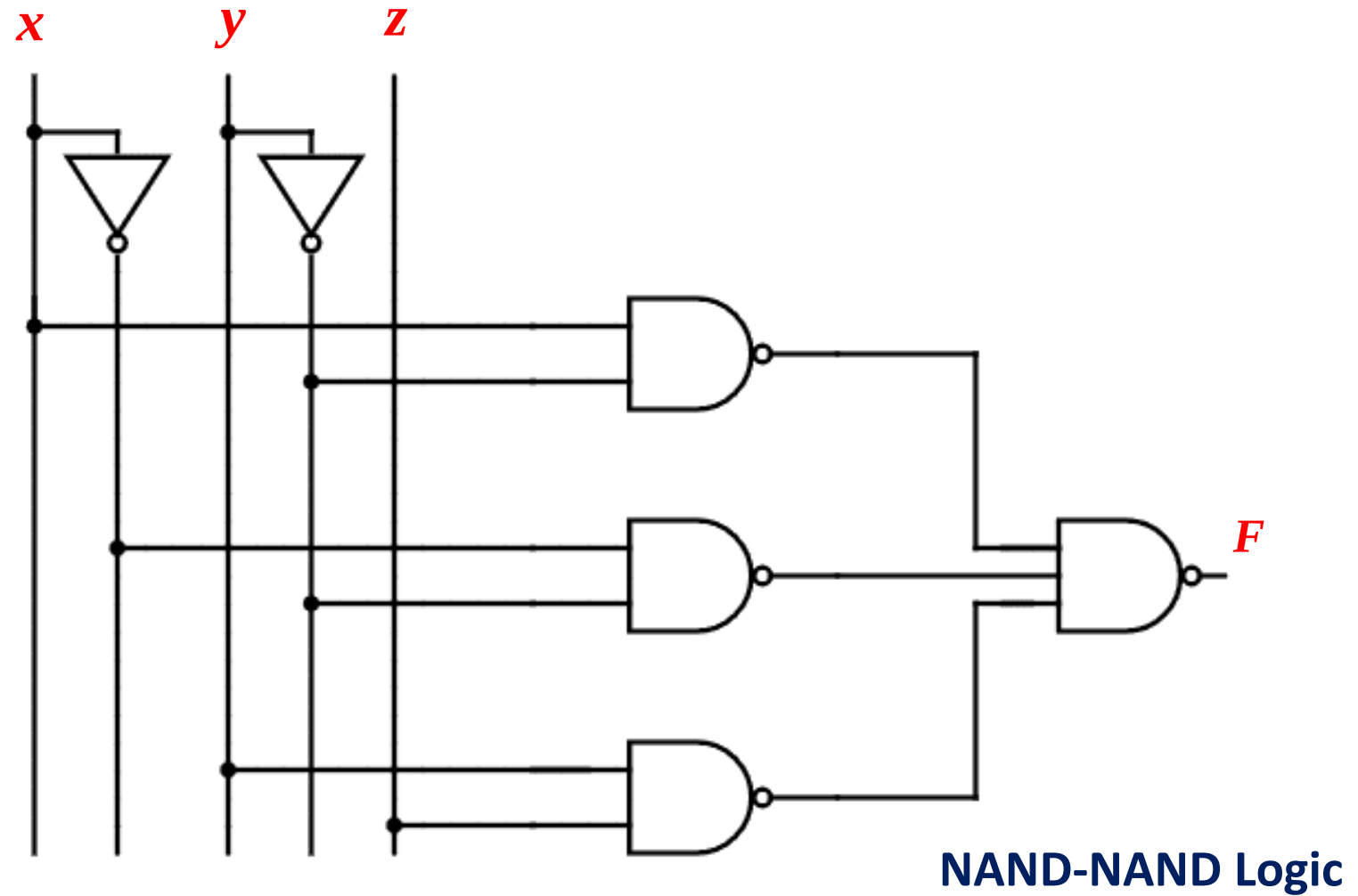
AND-OR Logic



NAND-NAND Logic

NAND IMPLEMENTATION

Method-2:



NAND IMPLEMENTATION

Multilevel NAND circuits

- ✓ The general procedure for converting a multilevel AND–OR diagram into an all-NAND diagram using mixed notation is as follows:
 1. Convert all AND gates to NAND gates with AND-invert graphic symbols.
 2. Convert all OR gates to NAND gates with invert-OR graphic symbols.
 3. Check all the bubbles in the diagram. For every bubble that is not compensated by another small circle along the same line, insert an inverter (a one-input NAND gate) or complement the input literal.

NAND IMPLEMENTATION

Example: Implement the following Boolean function using NAND gates

$$F = (AB' + A'B)(C + D')$$

Method-1:

$$\begin{aligned} F &= (AB' + A'B)(C + D') \\ &= AB'C + A'BC + AB'D' + A'BD' \end{aligned}$$

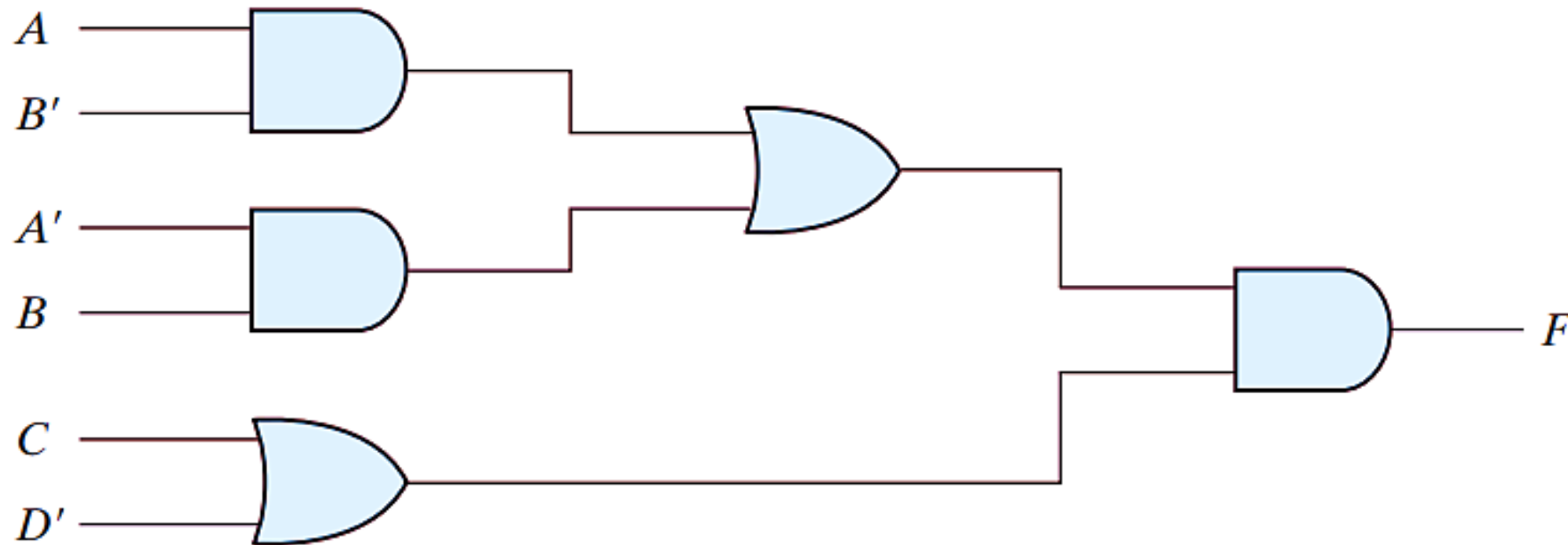
After converting the equation into SOP, we can implement using NAND gates.

NAND IMPLEMENTATION

Example: Implement the following Boolean function using NAND gates

$$F = (AB' + A'B)(C + D')$$

Method-2:



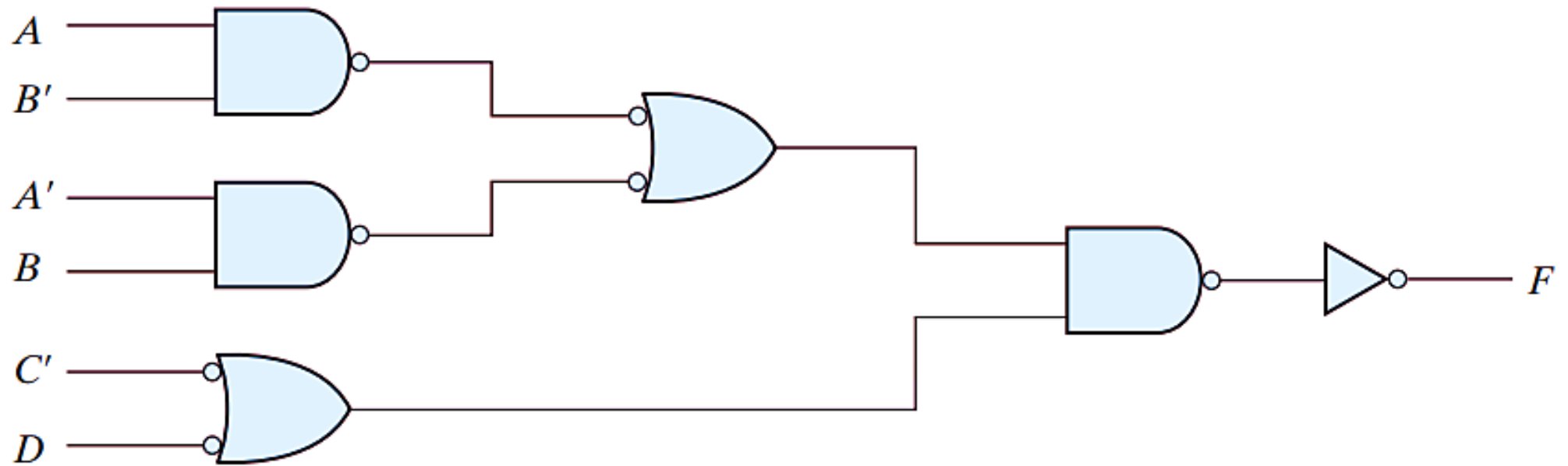
(a) AND-OR gates

NAND IMPLEMENTATION

Example: Implement the following Boolean function using NAND gates

$$F = (AB' + A'B)(C + D')$$

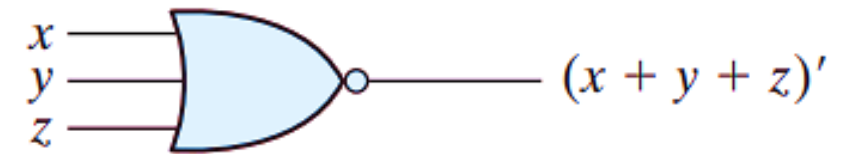
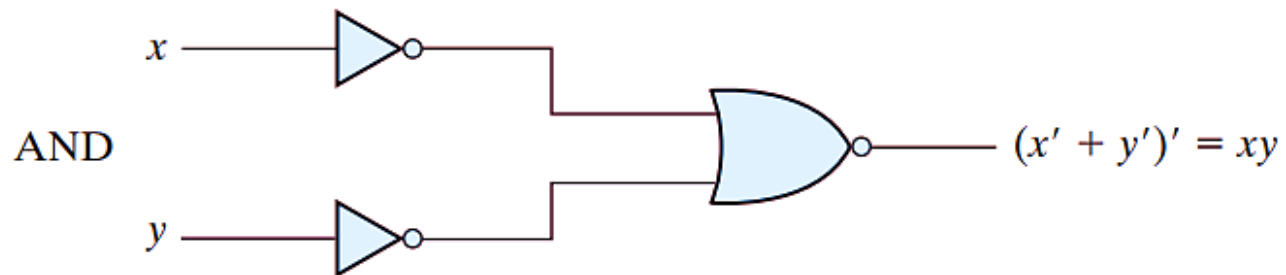
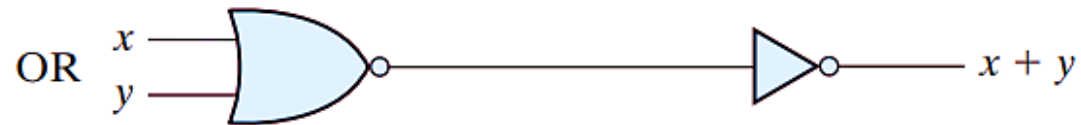
Method-2:



(b) NAND gates

NOR IMPLEMENTATION

- ✓ Any Boolean function, we need to show the basic logical operations, i.e., AND, OR and NOT operations in terms **NOR** alone.



(a) OR-invert



(b) Invert-AND

**Both (a) and (b) will perform
NOR operation**

NOR IMPLEMENTATION

Method-1:

- ✓ Simplify the Boolean function into product of sums (**POS**) form.
- ✓ Represent F in $(F')'$ form. Now each product term can be represented as NOR operation.

Example-1: Implement the following Boolean function using NOR gates

$$F = (A + B + C')(A + B)D$$

Solution:

Step-1: The function already in **POS form**

Step-2: $F = (F')' = [((A + B + C')(A + B)D)']' = [(A + B + C')' + (A + B)' + D']' = [x + y + z]'$

where, $x = (A + B + C')'$ & $y = (A + B)'$ & $z = D'$

NOR IMPLEMENTATION

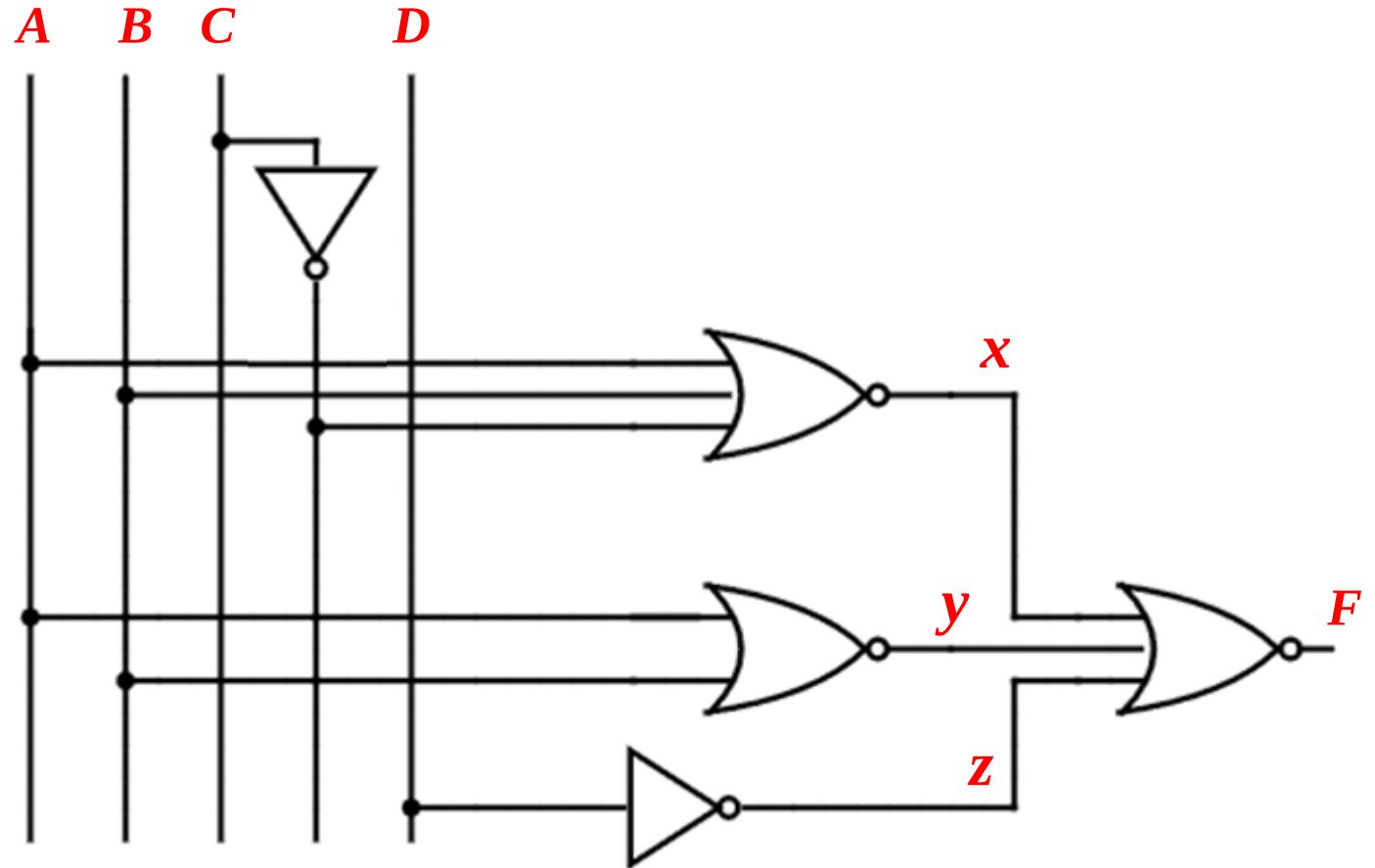
Example-1:

Step-3: Here,

$(A + B + C')'$ represents a NOR gate

$(A + B)'$ represents a NOR gate

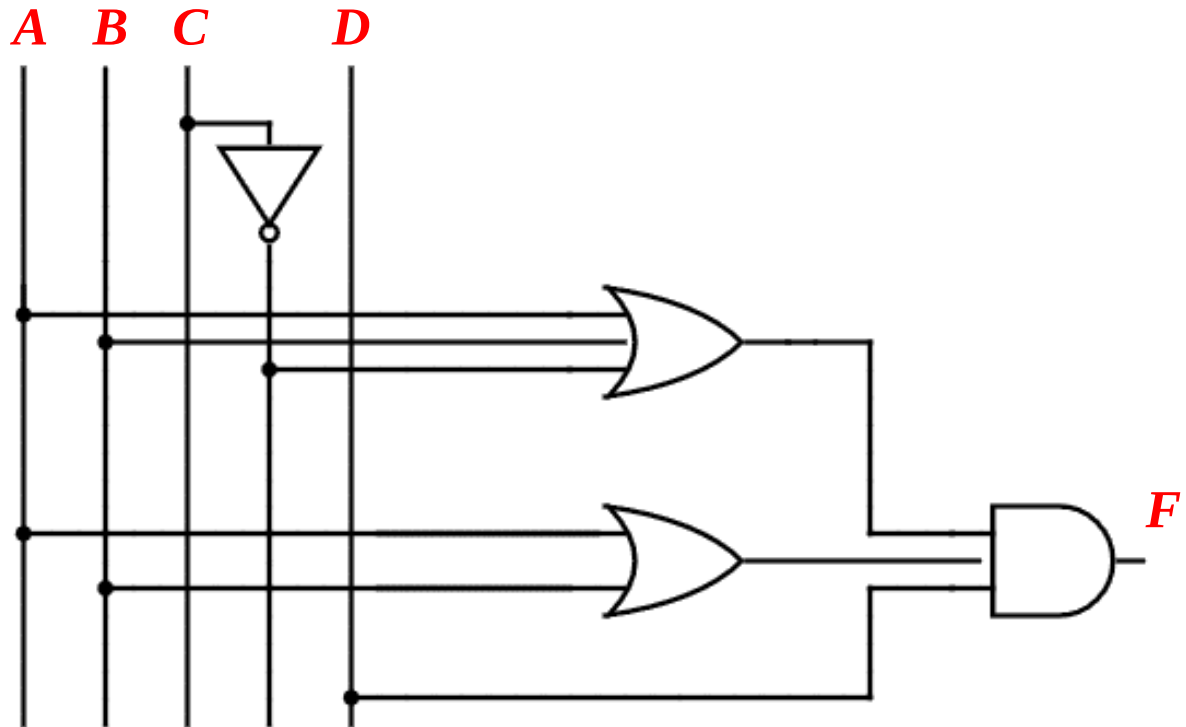
$[x + y + z]'$ represents a NOR gate



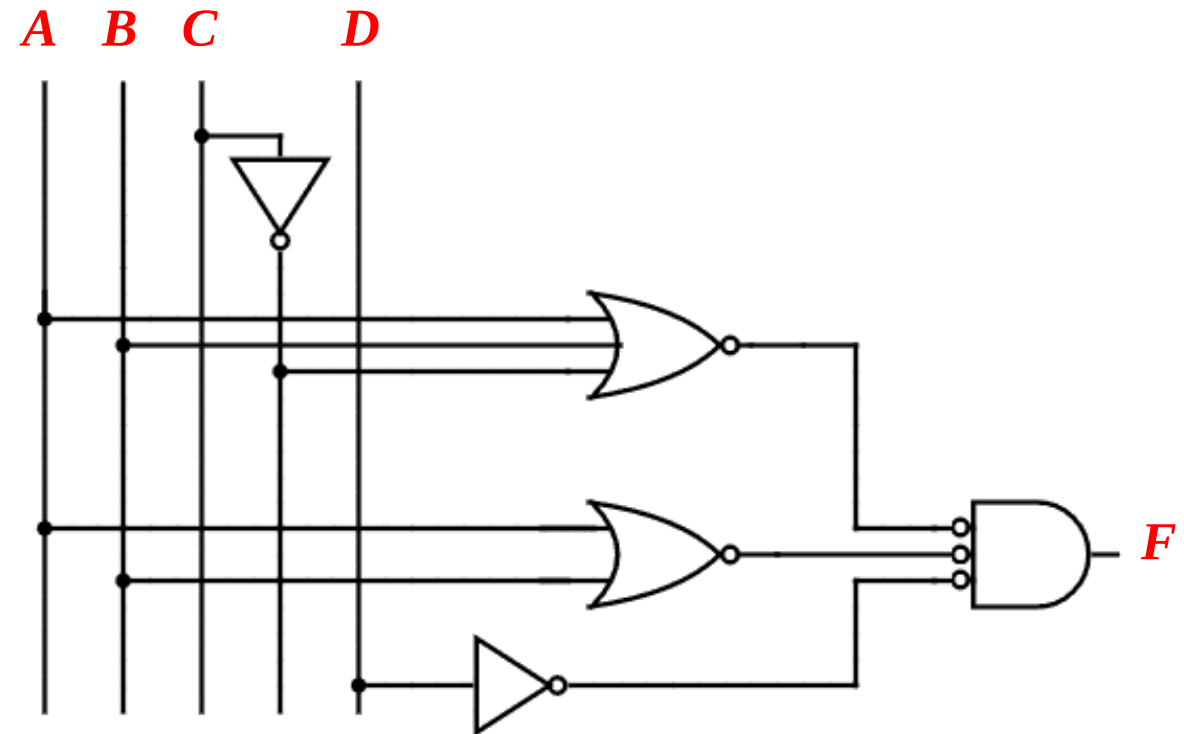
NOR-NOR Logic

NOR IMPLEMENTATION

Method-2:



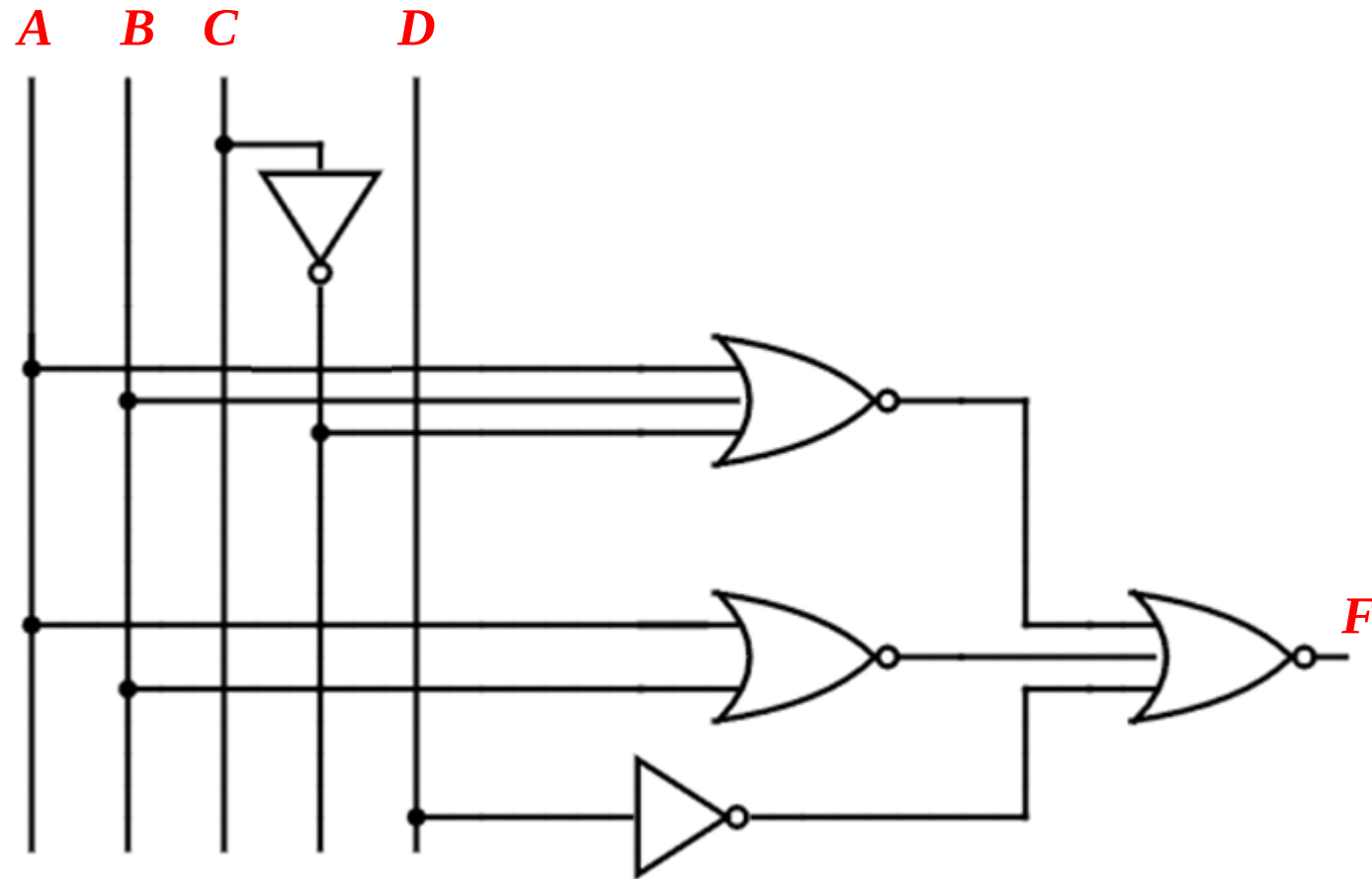
OR-AND Logic



NOR-NOR Logic

NOR IMPLEMENTATION

Method-2:



NOR-NOR Logic

NOR IMPLEMENTATION

Example-2: Implement the following Boolean function using NAND gates

$$F = \Sigma(2, 3, 4, 5, 7)$$

Solution:

Step-1: Obtain the SOP form

$$F' = \Sigma(0, 1, 6)$$

		yz			
		00	01	11	10
x	0	0	0		
	1				0

xyz'

$$F' = x'y' + xyz'$$

$$F = (x'y' + xyz')'$$

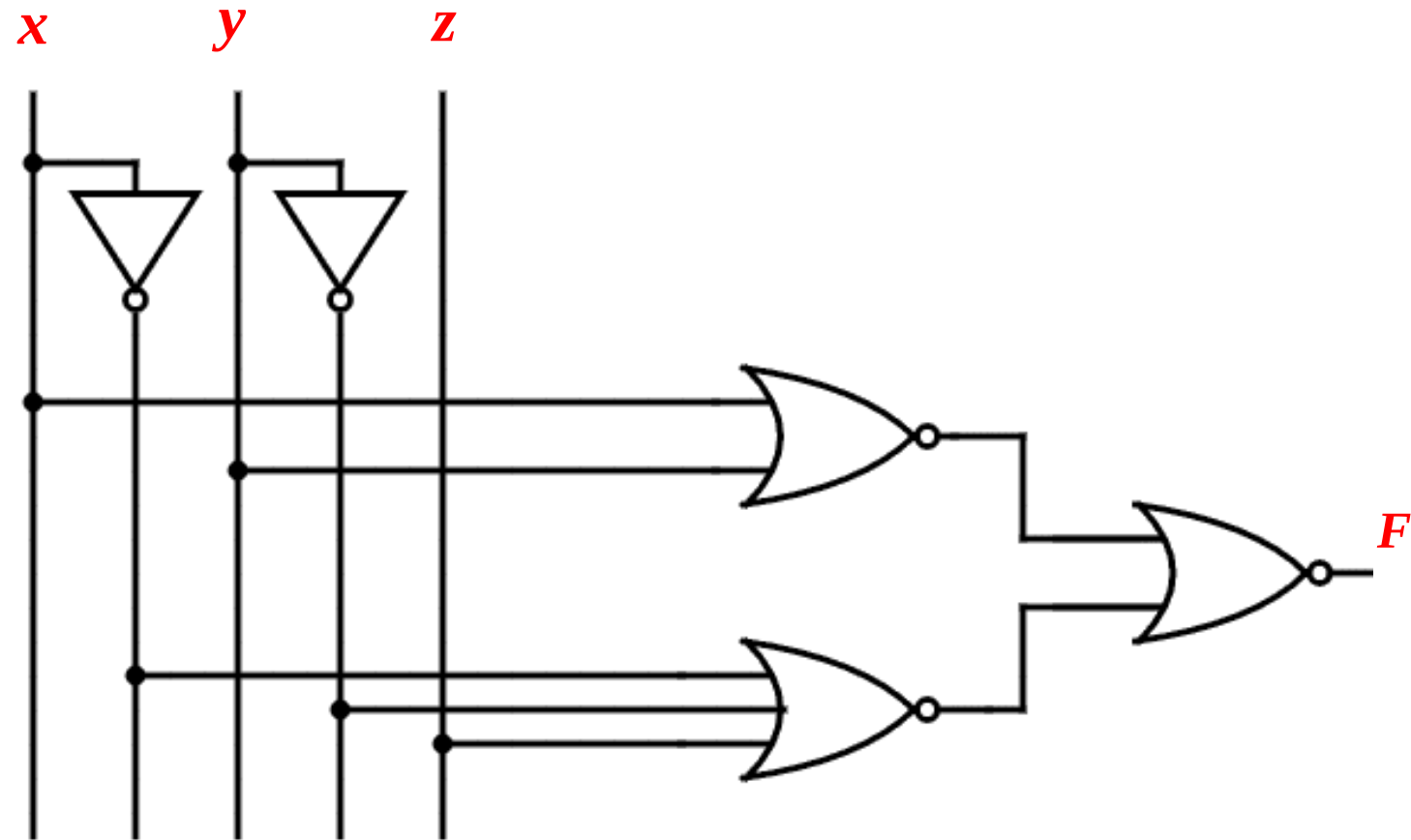
$$F = (x + y)(x' + y' + z)$$

Step-2:

$$F = (F')' = [(x + y)' + (x' + y' + z)']' = [A + B]'$$

NOR IMPLEMENTATION

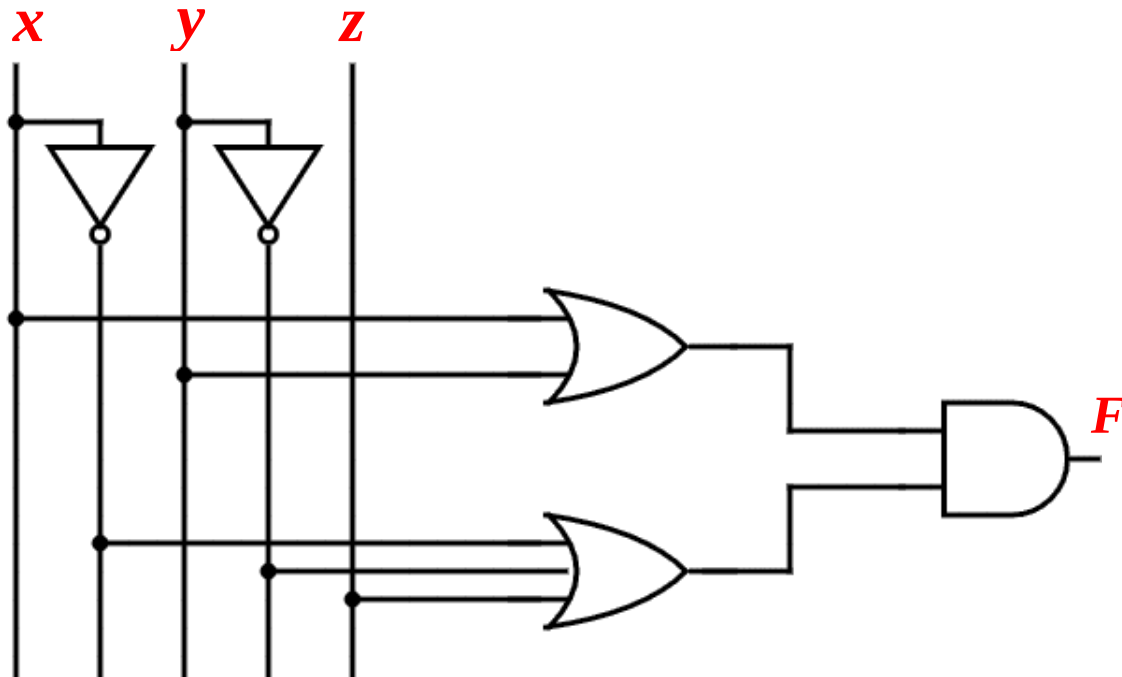
Step-3:



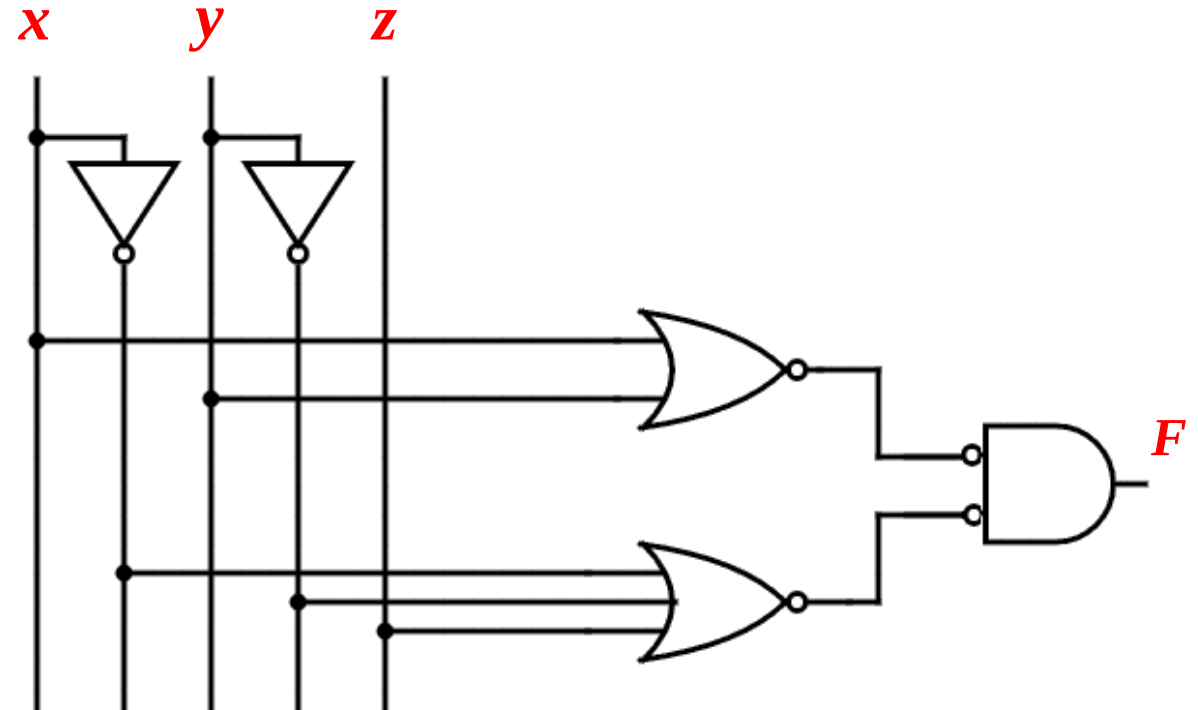
NOR-NOR Logic

NOR IMPLEMENTATION

Method-2:



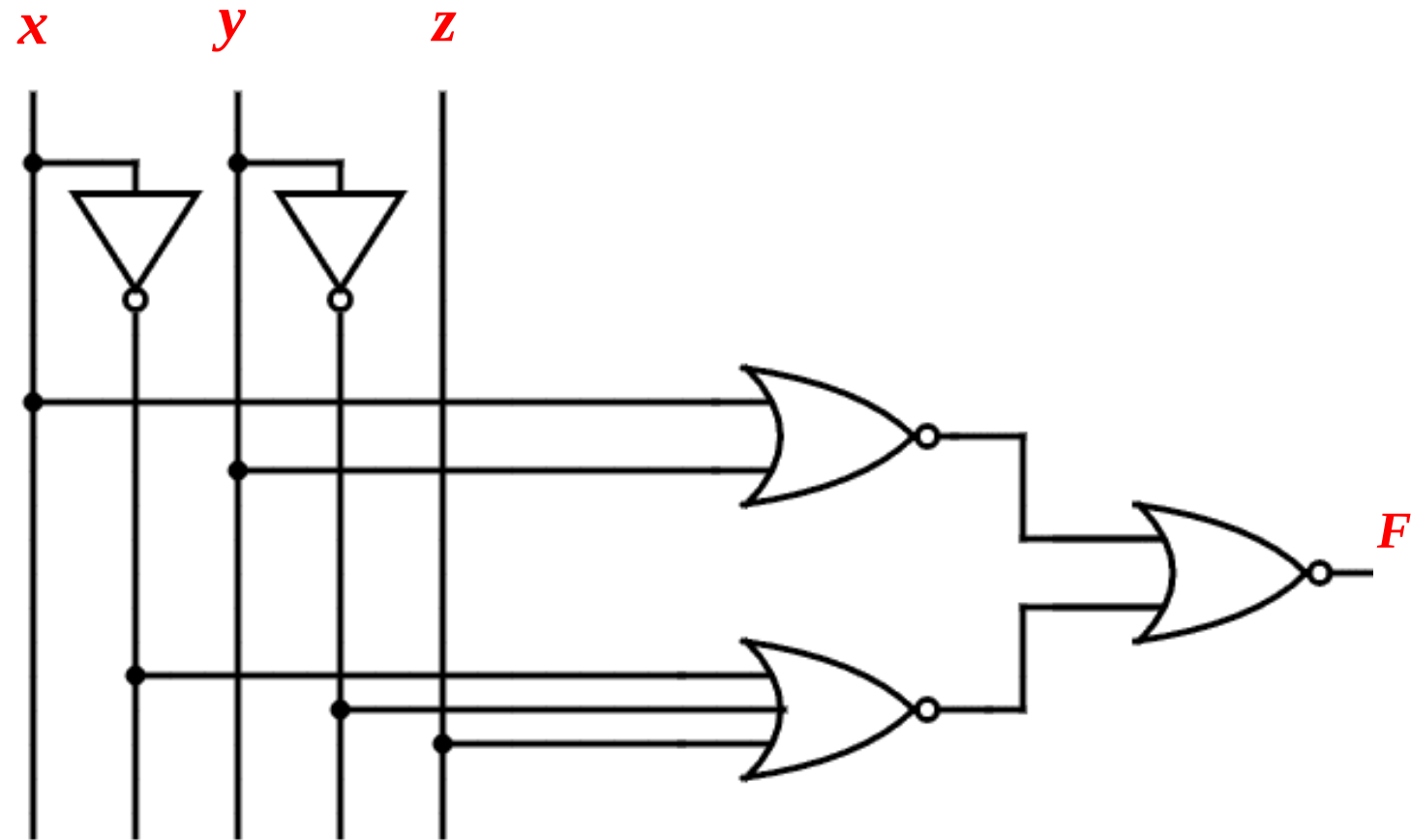
OR-AND Logic



NOR-NOR Logic

NOR IMPLEMENTATION

Method-2:



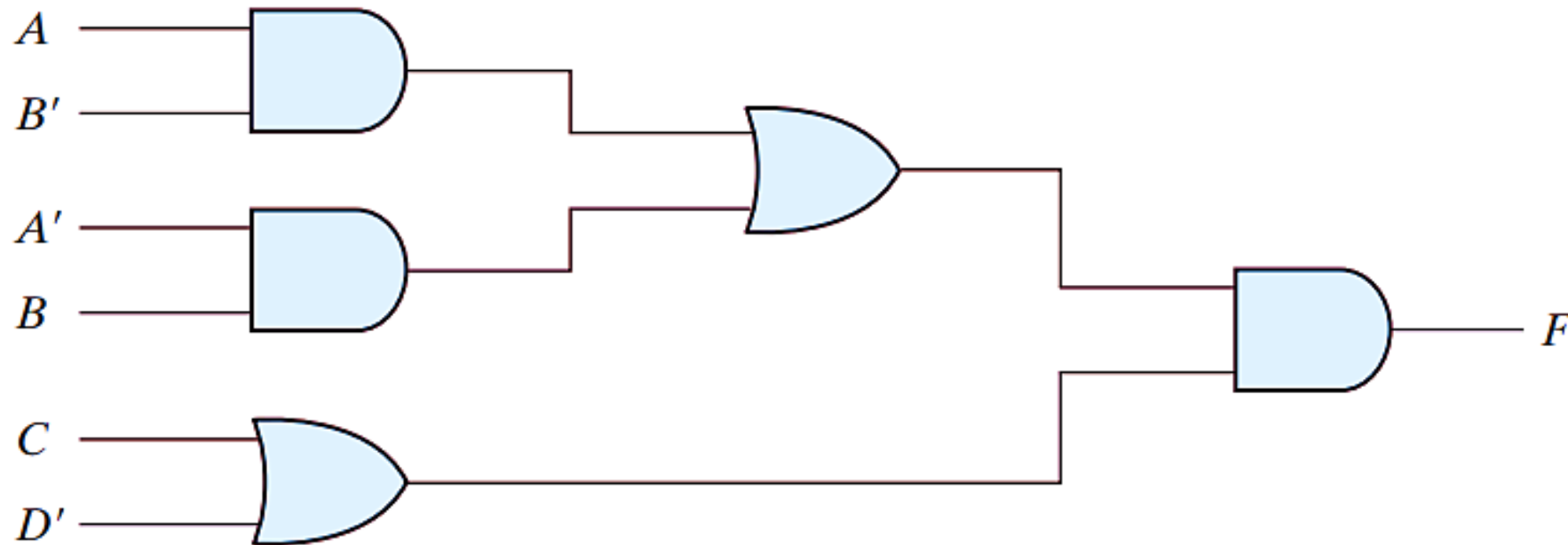
NOR-NOR Logic

NOR IMPLEMENTATION

Example: Implement the following Boolean function using NOR gates

$$F = (AB' + A'B)(C + D')$$

Method-2:



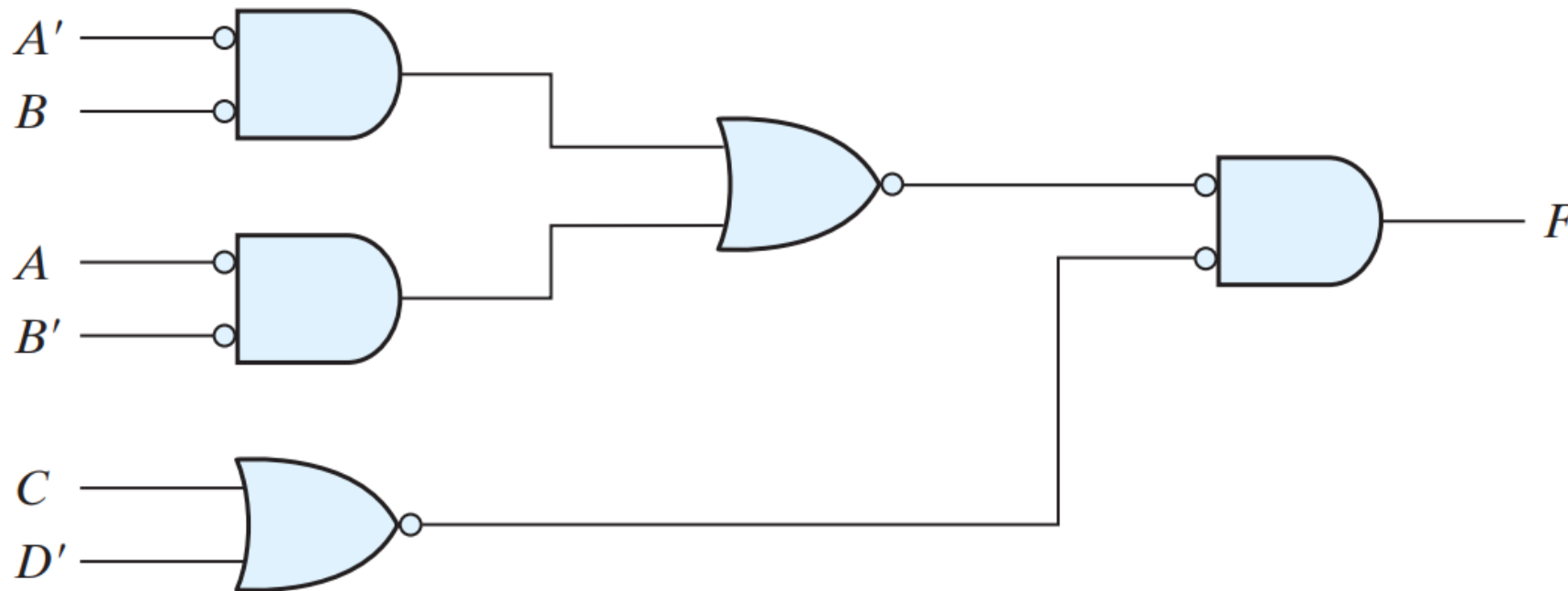
(a) AND-OR gates

NOR IMPLEMENTATION

Example: Implement the following Boolean function using NOR gates

$$F = (AB' + A'B)(C + D')$$

Method-2:



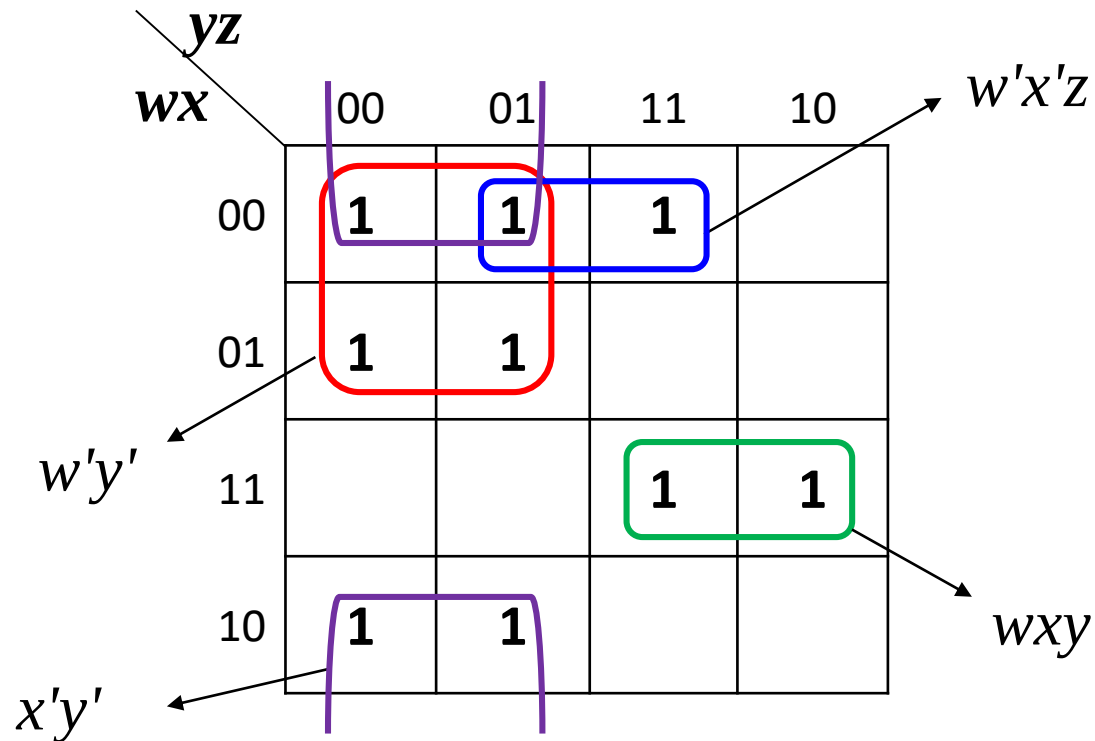
(b) NOR Gates

EXAMPLES

Example: Implement the following Boolean function using only NAND and only NOR gates

$$F = \Sigma (0, 1, 3, 4, 5, 8, 9, 14, 15)$$

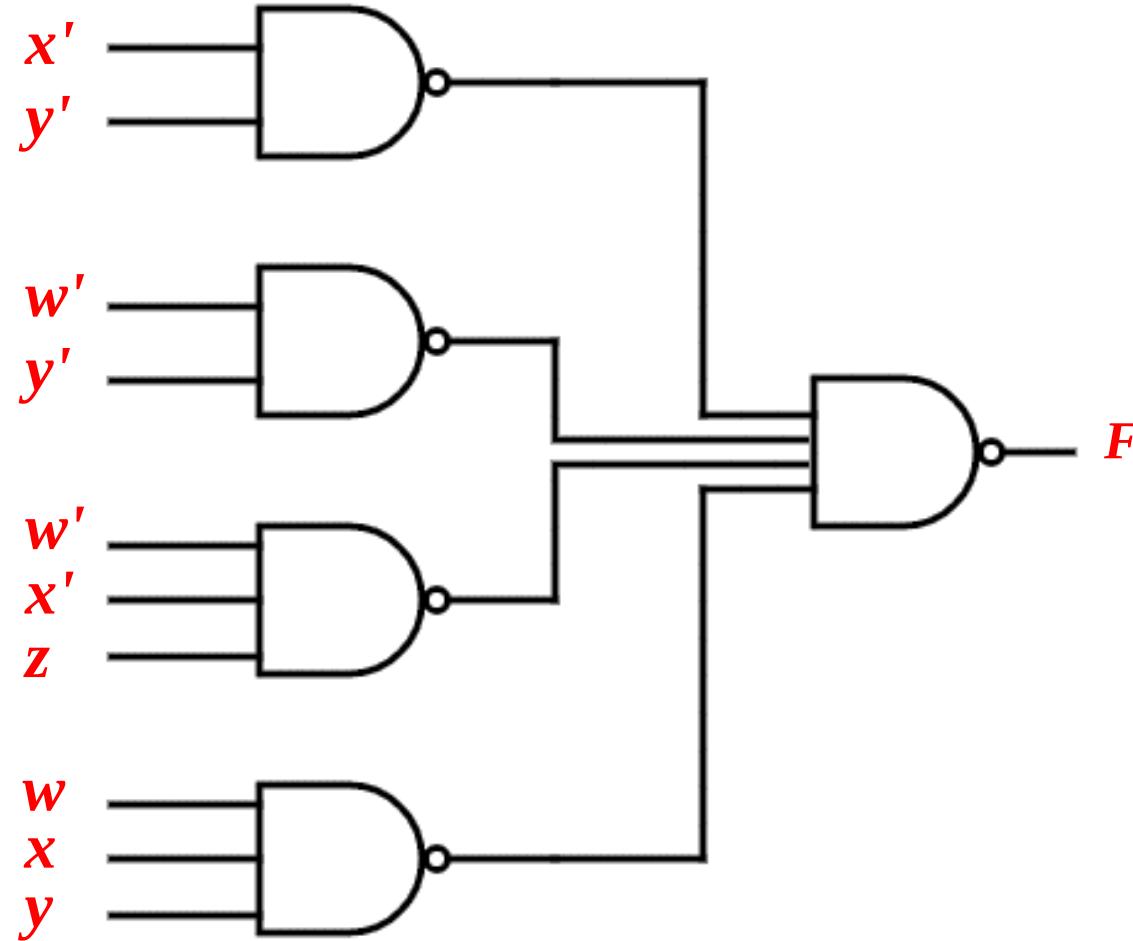
Solution: SOP: NAND implementation



$$F = x'y' + w'y' + w'x'z + wxy$$

EXAMPLES

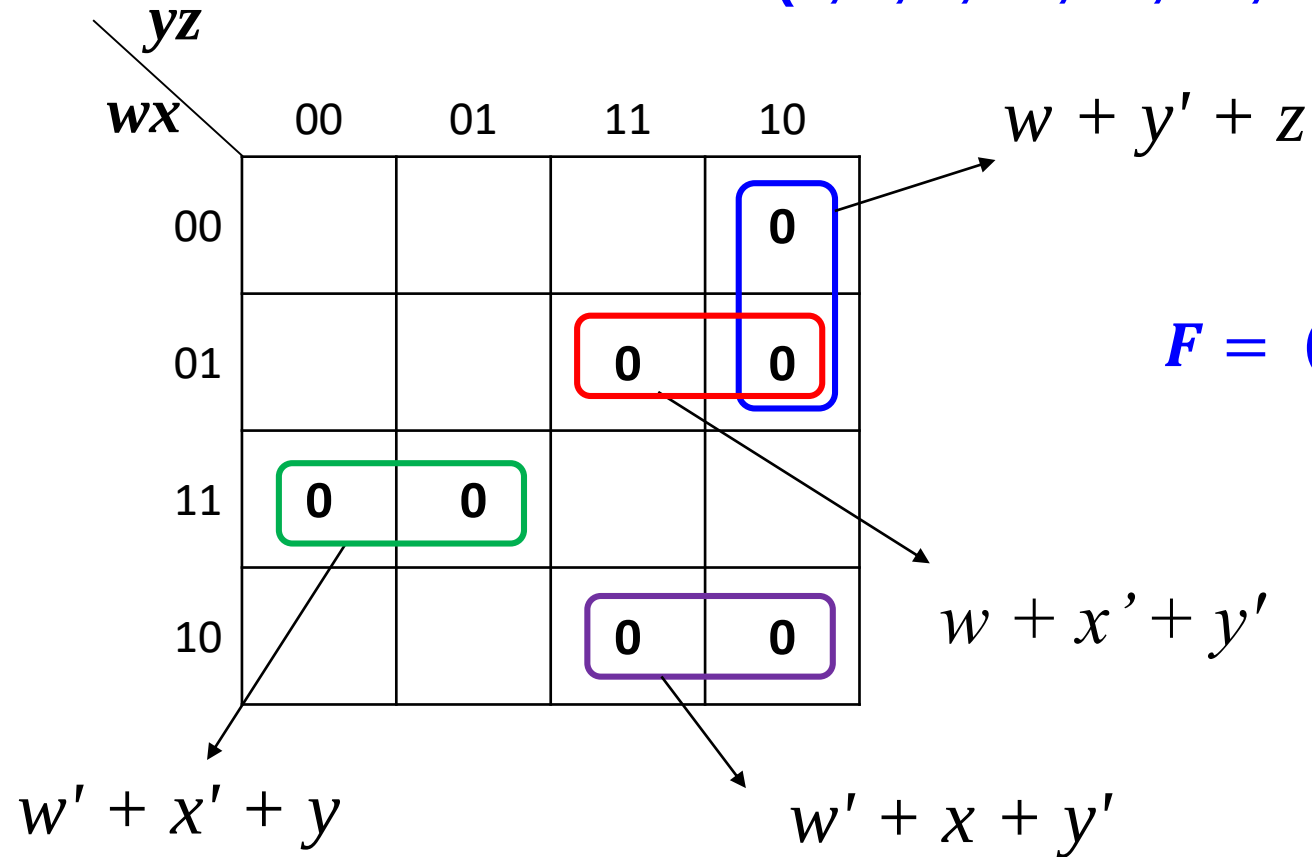
Solution: SOP: NAND implementation



EXAMPLES

Solution: POS: NOR implementation

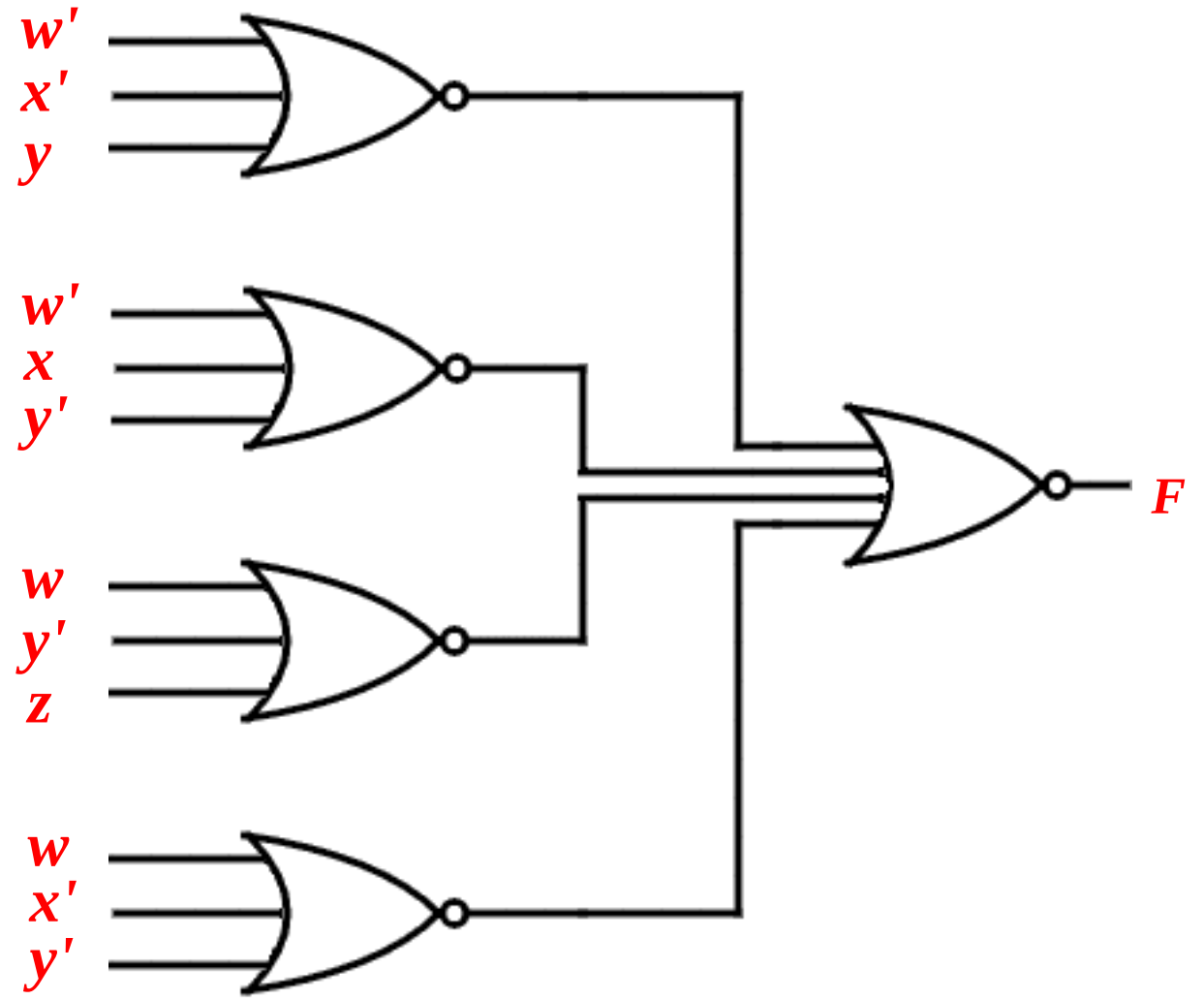
$$F = \Pi (2, 6, 7, 10, 11, 12, 13)$$



$$F = (w' + x' + y)(w' + x + y') \\ (w + x' + y')(w + y' + z)$$

EXAMPLES

Solution: POS: NOR implementation



Thank You